Recuperatorio 2do Parcial de Arquitectura y Organización del Computador - 2c2025

Normas generales

- El parcial es INDIVIDUAL
- El parcial es a libro abierto, pueden disponer de la bibliografía de la materia y acceder al repositorio de código del taller de system programming, desarrollado durante la cursada o a cualquier otro recurso que consideran. Sin embargo, **está totalmente prohibido el uso del celular, internet y chatbots.**
- El parcial es en los laboratorios, usando las computadoras de los laboratorios o sus propias computadoras, siempre que sean soportadas por el sistema de parciales detallado más adelante.
- Las resoluciones que incluyan código, pueden usar assembly o C. No es necesario que el código compile correctamente, pero debe tener un nivel de detalle adecuado para lo pedido por el ejercicio.
- Vamos a usar herramientas de detección de plagio para asegurarnos de que su entrega sea original. La entrega debe ser original para estar aprobada. El plagio en instancias de evaluación incurre sanciones que pueden incluir la suspensión académica.

Respecto al sistema de parciales:

- Deberán conectarse al sistema de parciales de la materia, el cual limitará su acceso a internet para que únicamente se puedan conectar a GitHub. Se debe **mantener la conexión** a este sistema hasta entregar el examen.
- Les proveeremos de un script que deberá ejecutarse al comienzo del parcial y no detenerse hasta haber entregado. Con él nos aseguramos que permanezcan conectados al sistema.
- Para aprobar el parcial deben implementar 2 de los 3 puntos del enunciado y que corran con éxito los tests funcionales, de abi y de memoria (valgrind).

Régimen de Aprobación

En este parcial se evalúa el manejo de los mecanismos de:

- Excepciones, interrupciones de hardware y syscalls mediante Interrupciones al procesador
- Manejo de memoria mediante Segmentación y Paginación
- Permisos y cambios de nivel de privilegio
- Administración del tiempo de procesamiento y del espacio de memoria para la ejecución de tareas.

Para la Arquitectura Intel x86. La entrega debe demostrar, en la medida exigida por el enunciado, comprensión clara de la mayoría estos mecanismos, e incomprensión de ninguno.

El parcial se aprueba con un puntaje numérico de **60 o más**, y califica para régimen de promoción con un puntaje numérico de **80 o más**.

NOTA: Lea el enunciado del parcial hasta el final antes de comenzar a resolverlo.

Régimen de Promoción (solo para quienes estén rindiendo por primera vez con ausencia justificada a la primer fecha)

Quien reciba un puntaje de **80 o más** en este parcial y tenga una calificación suficiente para promoción en el primer parcial podrá acceder al régimen de promoción. Esto es válido únicamente para la primer instancia de los parciales y no para sus recuperatorios.

Modalidad de Entrega

> [!WARNING] Se estará circulando por el aula una planilla escrita donde **deben** anotarse para que tengamos constancia de que su presencia en los laboratorios al momento del parcial. De no anotarse no podremos considerar su entrega como válida. La planilla, luego de circular, se encontrará en los escritorios al frente de cada laboratorio. **No se vayan sin anotarse!**

Deberán crear una nueva branch en este repositorio y allí desarrollar su solución en el archivo `Resolucion.md`, en formato Markdown (Acá tienen un machete de cómo dar formato en markdown). Es importante que no utilicen otro formato ya que dificultaría la corrección.

Si su solución utiliza código modificado del Trabajo Práctico, es importante que lo incluyan además de explicar en palabras las modificaciones (pueden copiar y pegar cualquier cosa de sus repositorios de TP). También pueden incluir imágenes si así lo desean. De tener dudas sobre qué funciones se pueden o no asumir implementadas, preguntar a los docentes durante el examen.

Una vez finalizado, completarán la entrega **creando un Pull Request e incluyendo a ayoc-bot como reviewer. Deben ver un mensaje de "Recibido!" de ayoc-bot para confirmar la entrega.** No es necesario poner la etiqueta `tests` en esta instancia.

Contexto

La nueva consola de videojuegos OrgaStation (donde se va a estrenar el esperado Half-Fantastruco 3) es en realidad una IBM PC con algo de marketing. Queremos agregar un subsistema gráfico a nuestro SO que sea lo suficientemente sofisticado para garantizar que no se vean en la pantalla “frames a medias” (también llamado screen tearing).



Para evitar ésto utilizaremos “double buffering”: además de la memoria de pantalla con la que ya trabajamos se agrega una pantalla “virtual” que se mantiene en la memoria RAM. Llamamos front buffer a la primera y back buffer a la segunda. Lo que éste mecanismo nos permite es dibujar en un lugar mientras el hardware de video copia los píxeles desde otro.

Justo antes de comenzar a dibujar una nueva imagen, el hardware de video notifica a nuestra consola por medio de la interrupción llamada `vblank` (vertical blanking). Esto le da al sistema un pequeño lapso de tiempo en el que copiar la información del back buffer al front buffer.

Dado el diseño de nuestro sistema no podemos prever cuándo seremos interrumpidos. **Cualquier syscall “lenta” [1] debe poder ser interrumpida** por el `vblank` durante su ejecución. Al responder a un vblank hay dos situaciones posibles: Si el

back buffer está actualmente en uso se debe incrementar el contador droppedFrames. Si no, se debe realizar la copia antes dicha.

El back buffer de nuestro sistema se encuentra en la dirección **física** `0xBE0BE000` y ocupa 65536 bytes (64KiB). El front buffer se encuentra en la dirección **física** `0xB8000` (y tiene el mismo tamaño). El vblank causa la interrupción número 41 en nuestro sistema.

Las tareas no manipularán el front buffer ni el back buffer directamente. Toda manipulación de la pantalla se realizará por medio de syscalls. Todos los accesos a memoria realizados por las syscalls será con paginación activa. La API gráfica a implementar consta de tres llamadas al sistema:

- `draw_start()`: Toma control de la pantalla. Si otra tarea ya la tiene, espera hasta poder adquirirla. Durante la ejecución del sistema sólo las tareas esperando control de la pantalla se encuentran "en pausa"; las demás deben ejecutarse normalmente.
    - Asumiendo que todas las tareas que piden la pantalla en algún momento la liberan, las tareas que quedan esperando la pantalla deben eventualmente recibirla.
- `draw_primitive(type, point, data)`: Dibuja una figura geométrica en el back buffer. Utiliza la función de sistema `draw_primitive_to(dest, type, point, data)` la cuál es lenta. El parámetro `dest` de `draw_primitive_to` es la dirección virtual del back buffer. Si la tarea que invoca `draw_primitive` no tiene el control actual de la pantalla el sistema debe ignorar la solicitud.
- `draw_end()`: Libera el control de la pantalla. Esto permite que eventualmente otras tareas lo tomen por medio de `draw_start`. Si la tarea que invoca `draw_end` no tiene el control actual de la pantalla el sistema debe ignorar la solicitud.

---

[1] Definimos que una syscall es "lenta" cuando su rutina de atención potencialmente requiere procesar durante varios ciclos de clock enteros para completarse.

Ejercicio 1 (15 puntos)

Describir los cambios necesarios para **registrar** las syscalls en el sistema. Las rutinas de atención se implementarán en puntos posteriores.

Además, responder:

- ¿Cómo se puede lograr que las syscalls "lentas"[1] puedan ser interrumpidas?
- ¿Cuáles de las nuevas syscalls deben ser *interrumpibles*?
- ¿Qué eventos pueden causar esas interrupciones?

Ejercicio 2 (25 puntos)

El sistema debe permitir que el kernel tenga acceso al *back buffer* en todo momento, mientras que el código de usuario no debe poder escribir de manera directa en el *back buffer*. ¿Cómo se debe modificar el sistema para lograr esto? En base a las modificaciones propuestas, ¿cuál es la dirección virtual del *back buffer*? Recordar que el *back buffer* ocupa 64KiB.

Ejercicio 3 (15 puntos)

Describir los cambios al sistema necesarios para implementar

```
void draw_primitive(uint32_t type, uint32_t point, uint32_t data)
```

asumiendo resuelta la función `void draw_primitive_to(vaddr dest, uint32_t type, uint32_t point, uint32_t data)`.

Además, se puede tomar como dada la siguiente función, que de usarse deberá ser implementada en un ejercicio posterior. Su uso no es obligatorio pero es **fuertemente** recomendado:

- `task_id dueño_pantalla()`: devuelve el `task_id` de la tarea que tiene el control de la pantalla actualmente, o -1 si la pantalla no tiene dueño.

Recuerden:

- Si la tarea que invoca la *syscall* no tiene el control actual de la pantalla el sistema debe ignorar la solicitud.
- La definición de `draw_primitive` de más arriba es la firma de la syscall, con la que informamos qué recibe y que devuelve la syscall.No es requisito que implementen una función de esa pinta en el sistema.
- `task_id` es un renombre de `uint8_t`.

Ejercicio 4 (20 puntos)

Explicar todas las modificaciones al sistema necesarias para implementar la interrupción de hardware asociada al `vblank`. Esta interrupción debe:

- Revisar si el *back buffer* está listo para ser copiado.
- Si lo está, realizar la copia del *back buffer* al *front buffer*.
- Si no lo está, aumentar el contador `droppedFrames` . Comentar cómo se inicializa esta variable y dónde se declara.

Se puede asumir resuelta la siguiente función, que de usarse debe ser implementada en un ejercicio posterior. Su uso no es obligatorio pero es fuertemente recomendado:

- `bool está_pantalla_lista()` que indica si el *back buffer* está listo para ser copiado al *front buffer* o no (es decir: si **ninguna tarea controla la pantalla**).

Ejercicio 5 (20 puntos)

Implementar `draw_start` y `draw_end`, asumiendo los ejercicios anteriores bien resueltos.

Si una tarea invoca a `draw_end` sin tener control de la pantalla el sistema debe ignorar esa solicitud.

Asumiendo que todas las tareas que piden la pantalla en algún momento la liberan, la solución debe garantizar que las tareas que quedan esperando la pantalla eventualmente tomen control de la misma.

Se pueden tomar como dadas las siguientes funciones, que de usarse deberán ser implementadas en el próximo ejercicio. Su uso no es obligatorio pero es fuertemente recomendado:

- `bool tomar_control_pantalla()`: le da control de la pantalla a la tarea actual (si ninguna tarea la controla). Si logra su cometido devuelve `true`, si no `false` (otra tarea ya tenía el control).
- `bool liberar_pantalla()`: libera la pantalla si quién la llama es la tarea que tiene control de la pantalla actualmente. Si logra su cometido devuelve `true`, si no `false` (la tarea actual no tenía el control).
- `task_id dueño_pantalla()`: devuelve el `task_id` de la tarea que tiene el control de la pantalla actualmente, o -1 si la pantalla no tiene dueño.

Pueden asumir que estas funciones no son interrumpibles, independientemente del contexto en que son llamadas. No hace falta explicar por qué no son interrumpibles.

Recuerden que `task_id` es un renombre de `uint8_t`.

Ejercicio 6 (5 puntos)

Implementar las funciones auxiliares que fueron utilizadas anteriormente. Para recapitular, las funciones provistas son:

- `bool está_pantalla_lista()`
- `bool tomar_control_pantalla()`
- `bool liberar_pantalla()`
- `task_id dueño_pantalla()`

No es necesario explicar cómo se logra que no sean interrumpibles. En caso de querer hacerlo de todos modos "correctamente" pueden ayudarse con las instrucciones `pushfd` y `popfd` [2].

---

[2] Las entradas del manual de pushfd y popfd son las siguientes (recortadas a lo principal). Ante cualquier duda pueden consultar con los docentes por el funcionamiento de estas instrucciones.

PUSHF/PUSHFD/PUSHFQ — Push EFLAGS Register Onto the Stack
Decrements the stack pointer by 4 (if the current operand-size attribute is 32) and pushes the entire contents of the EFLAGS register onto the stack, or decrements the stack pointer by 2 (if the operand-size attribute is 16) and pushes the lower 16 bits of the EFLAGS register (that is, the FLAGS register) onto the stack. These instructions reverse the operation of the POPF/POPFD instructions.

POPF/POPFD/POPFQ — Pop Stack Into EFLAGS Register
Pops a doubleword (POPFD) from the top of the stack (if the current operand-size attribute is 32) and stores the value in the EFLAGS register, or pops a word from the top of the stack (if the operand-size attribute is 16) and stores it in the lower 16 bits of the EFLAGS register (that is, the FLAGS register). These instructions reverse the operation of the PUSHF/PUSHFD/PUSHFQ instructions.